

# Deep Learning Methods for Encrypted Traffic Analysis and Classification in TLS & QUIC Protocols

Yasin Alperen Bozkurt

September 17, 2026

# Contents

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                          | <b>3</b>  |
| <b>2</b> | <b>Fundamental Concepts</b>                                  | <b>4</b>  |
| 2.1      | Fundamentals of Network Communication . . . . .              | 4         |
| 2.1.1    | Client-Server Model and Round-Trip Time (RTT) . . . . .      | 4         |
| 2.1.2    | The Handshake Process . . . . .                              | 4         |
| 2.2      | TLS and QUIC Protocols . . . . .                             | 4         |
| 2.2.1    | The TLS Protocol . . . . .                                   | 4         |
| 2.2.2    | The QUIC Protocol and Its Features . . . . .                 | 4         |
| 2.3      | Traffic Representation . . . . .                             | 5         |
| 2.3.1    | Flow-Level Statistical Features . . . . .                    | 5         |
| 2.3.2    | Packet-Level Temporal Features . . . . .                     | 5         |
| 2.3.3    | Byte-Level Representations . . . . .                         | 5         |
| <b>3</b> | <b>Literature Review</b>                                     | <b>6</b>  |
| 3.1      | Flow-Based Statistical Models . . . . .                      | 6         |
| 3.2      | Payload-Based Deep Learning (CNN) . . . . .                  | 6         |
| 3.3      | Sequential Models (RNN/LSTM/GRU) . . . . .                   | 6         |
| 3.4      | Transformer-Based Approaches . . . . .                       | 6         |
| 3.4.1    | Encrypted Traffic BERT . . . . .                             | 6         |
| 3.4.2    | Multi-Instance Encrypted Traffic Transformer . . . . .       | 7         |
| 3.5      | Transfer Learning and Foundation Models . . . . .            | 7         |
| 3.5.1    | QUIC Domain Recognition Pre-training . . . . .               | 7         |
| 3.6      | Graph-Based Methods and Graph Drift . . . . .                | 7         |
| 3.6.1    | Mixture-of-Experts Enhanced Detection . . . . .              | 7         |
| 3.7      | Comparative Performance Analysis . . . . .                   | 7         |
| <b>4</b> | <b>Open Problems and Limitations</b>                         | <b>7</b>  |
| 4.1      | Concept Drift and Temporal Distribution Shift . . . . .      | 7         |
| 4.2      | Zero-Day Traffic Detection . . . . .                         | 8         |
| 4.3      | Rare Class Classification Problem . . . . .                  | 8         |
| 4.4      | Real-Time Processing Overhead . . . . .                      | 8         |
| 4.5      | Adversarial Vulnerability . . . . .                          | 8         |
| <b>5</b> | <b>Future Research Directions</b>                            | <b>9</b>  |
| 5.1      | Privacy-Preserving Training via Federated Learning . . . . . | 9         |
| 5.2      | Few-Shot and Meta-Learning Approaches . . . . .              | 9         |
| <b>6</b> | <b>Conclusion</b>                                            | <b>10</b> |

# 1 Introduction

In the early 2000s, when Internet traffic was transmitted unencrypted as plain text, packet analysis was straightforward. However, sending packets in the clear created a major security vulnerability. For this reason, protocols such as Transport Layer Security (TLS) and Google’s Quick UDP Internet Connections (QUIC) were developed to encrypt Internet traffic. With the encryption of packets, the traditional packet analysis methods used to inspect malicious traffic became insufficient. As a result, deep learning-based approaches have come to the forefront for classifying encrypted Internet traffic.

This study reviews the literature published between 2020 and 2026 on deep learning-based methods for encrypted traffic analysis. Transformer-based architectures (ET-BERT, MIETT), transfer learning strategies, and graph-based methods are examined. The performance of these approaches, which are essential for detecting and blocking malicious traffic, is compared, and the key limitations of existing approaches are identified. This study also examines which methods are best suited to which types of traffic.

This study is a literature review of deep learning-based encrypted traffic analysis and classification methods for the TLS and QUIC protocols. The main objectives of the study are to: review the proposed deep learning approaches (CNN, RNN, LSTM, Transformer), compare traffic representation methods (flow-based, packet-based, byte-level), analyze the limitations of existing solutions (concept drift, adversarial attacks, zero-day traffic), and propose future research directions grounded in the literature findings.

Encrypted traffic analysis is needed in the following areas:

**Network Security:** Detecting malicious activity (ransomware, C&C communication, data exfiltration) carried out over encrypted channels [5].

**Quality of Service (QoS):** Correctly prioritizing encrypted video streaming, VoIP, and gaming traffic [4].

**Network Operations:** Supporting traffic engineering, capacity planning, and anomaly detection processes [3].

**Compliance and Monitoring:** Encrypted network traffic must be analyzed in order to ensure policy compliance and monitor network usage in enterprise networks.

Motivated by these needs, a systematic analysis of the current literature was carried out and this report was prepared.

## 2 Fundamental Concepts

This section introduces the fundamental concepts and technical background required to understand encrypted traffic analysis and classification.

### 2.1 Fundamentals of Network Communication

#### 2.1.1 Client-Server Model and Round-Trip Time (RTT)

Network communication takes place between a client and a server. **Round-Trip Time (RTT)** refers to the total time required for a message to travel from the client to the server and for its response to return.

#### 2.1.2 The Handshake Process

In a secure network communication, the client and server go through a negotiation process called a **handshake**. This process includes the following:

Steps such as authentication, selection of the encryption algorithm, and key exchange take place during the handshake process.

### 2.2 TLS and QUIC Protocols

#### 2.2.1 The TLS Protocol

Transport Layer Security (TLS) is the most widely used standard for encrypting communication over the Internet. TLS has three core functions: (1) data confidentiality through **encryption**, (2) confirmation of server/client identity through **authentication**, and (3) a guarantee that data has not been tampered with through **integrity**.

With TLS 1.3, Server Name Indication (SNI) and certificate information (Encrypted Client Hello - ECH) were encrypted to prevent domain information from being exposed, and the handshake process was optimized. These changes have rendered domain-based classification infeasible.

#### 2.2.2 The QUIC Protocol and Its Features

The QUIC (Quick UDP Internet Connections) protocol, designed by Google to replace TLS and TCP, was standardized as RFC 9000 in 2021.

- **Running Over UDP:** Eliminates the head-of-line blocking problem inherent in TCP's ordered packet delivery, allowing each stream to progress independently of the others.
- **Multiplexing:** Enables independent streams to be created over a single connection, allowing video, audio, and data packets to be transmitted simultaneously.
- **Connection Migration:** Preserves the connection across IP address changes, which is important for mobile devices.
- **Fully Encrypted Headers:** Encrypts headers in addition to the payload, increasing data confidentiality. However, this makes analysis methods more difficult.

Because data in the QUIC protocol is fully encrypted, flow-based analysis methods are made more difficult. However, information such as packet sizes and inter-packet timing patterns can still be used for analysis [4].

## 2.3 Traffic Representation

Since the payload is encrypted in encrypted traffic, models operate on **side-channel** information. Three main approaches stand out in the literature:

### 2.3.1 Flow-Level Statistical Features

Statistical features extracted from a network flow using tools such as CICFlowMeter [9] are used. These features include: packet count, byte count, duration, packet size statistics, inter-arrival time statistics, flag distributions (TCP: SYN, ACK, FIN, RST), window size features, and forward/backward direction-based features.

### 2.3.2 Packet-Level Temporal Features

This approach attempts to analyze the sequential patterns of packets by preserving their order. Sequential features such as packet length sequence, inter-arrival time sequence, direction sequence, and timestamp sequence are used.

RNN/LSTM- and Transformer-based models can capture these features through their memory mechanisms. State-of-the-art methods such as ET-BERT [1] and MIETT [2] employ this approach.

### 2.3.3 Byte-Level Representations

In this approach, analysis is carried out using the raw byte values of packets. Even when the payload is encrypted, features such as the hexadecimal or integer representation of the first  $N$  bytes, byte bigram/trigram features, byte value histograms, and payload entropy can be used for analysis.

## 3 Literature Review

This section reviews, compares, and classifies the deep learning approaches used for classifying encrypted network traffic.

### 3.1 Flow-Based Statistical Models

Traditional machine learning methods such as Random Forest, XGBoost, and SVM use features extracted via tools like CICFlowMeter [9]. These methods are easy to compute, but they cannot capture or interpret sequential packet patterns. The FlowCLIP [4] study, conducted on the CESNET-QUIC22 dataset, observed an 11.83% performance drop within two weeks.

### 3.2 Payload-Based Deep Learning (CNN)

One-Dimensional Convolutional Neural Networks are used to capture patterns within packets. These methods capture such patterns and, because their computations are performed in parallel, they can produce inferences in short amounts of time.

The Hardware-Aware NAS-CNN [10] study used Neural Architecture Search to achieve 96.60% accuracy. The resulting model has only 88,000 parameters – 444 times fewer than other CNN-based methods. However, because CNNs require fixed-size inputs, they are unable to interpret sequential packet patterns.

### 3.3 Sequential Models (RNN/LSTM/GRU)

Recurrent neural networks such as RNN, LSTM, and GRU are used to interpret sequential packet patterns. Thanks to their memory mechanisms, these methods can make sense of packets arriving in sequence. Because they process data sequentially, they cannot be parallelized, which leads to problems such as high latency. In addition, traditional methods (K-NN, decision trees) can be more resilient against DeepFool attacks than deep learning models [6].

### 3.4 Transformer-Based Approaches

Since 2022, Transformer architectures have become the dominant approach in the field of encrypted traffic classification. The self-attention mechanism underlying Transformers allows every packet in a sequence to interact directly with every other packet. This resolves both the receptive-field limitation of CNNs and the sequential-processing limitation of RNNs. Transformers can capture long-range dependencies while also enabling parallel processing.

#### 3.4.1 Encrypted Traffic BERT

Lin et al. [1] adapt the BERT architecture to the network traffic domain, modeling packets as tokens and flows as sentences. With this approach, they draw an analogy between the word-sentence relationship in natural language processing and the packet-flow relationship. In their performance evaluations, they achieved an F1 score of 99.2% on the ISCX-Tor dataset (a 4.4 percentage-point absolute improvement) and an F1 score of 97.4% on the CSTNET-TLS 1.3 dataset (a 10.0 percentage-point absolute improvement).

### 3.4.2 Multi-Instance Encrypted Traffic Transformer

The Multi-Instance Encrypted Traffic Transformer (MIETT) [2] combines token-level attention (within a packet) and flow-level attention (across packets) through a Two-Level Attention mechanism. It employs novel pre-training tasks, Packet Relative Position Prediction (PRPP) and Flow Contrastive Learning (FCL). It achieved 93.00% accuracy on the CrossPlatform dataset, an 8.37 percentage-point improvement over ET-BERT, and 88.53% accuracy on the CICIoT dataset, a 5.24 percentage-point improvement over ET-BERT.

## 3.5 Transfer Learning and Foundation Models

### 3.5.1 QUIC Domain Recognition Pre-training

The IEEE TNSM 2026 study [3] proposes a universal embedding function by pre-training on QUIC traffic using domain recognition. Metric learning is applied via ArcFace loss, followed by fine-tuning for the target tasks. As a result, the study achieved the best performance on 9 of 10 target tasks, with an average improvement of 6.4%. Using transfer learning, it reached 93.00% accuracy on the CrossPlatform dataset.

## 3.6 Graph-Based Methods and Graph Drift

### 3.6.1 Mixture-of-Experts Enhanced Detection

Mixture-of-Experts Enhanced Detection [5] models network traffic as a graph representation, in which each IP address is represented as a node and the network flows between IP addresses as edges. This study introduces the concept of **Graph Drift**, which is divided into Graph Scale Drift and Flow Statistics Drift. In the Mixture-of-Experts architecture, each expert module is specialized for a different drift type, and a Gating Network performs dynamic expert selection. In the performance evaluation, MalMoE achieved an accuracy of 0.9716 and an F1 score of 0.9479, while the best baseline method, E-GraphSAGE, achieved an accuracy of 0.7172 and an F1 score of 0.6318.

## 3.7 Comparative Performance Analysis

This section presents a comparative evaluation of the performance of the main deep learning methods reviewed in the field of encrypted traffic classification. Table 1 summarizes the leading methods in the literature in terms of their architectures, the datasets used, the performance results achieved, and their key disadvantages.

# 4 Open Problems and Limitations

This section analyzes the fundamental limitations faced by existing deep learning approaches and explains why they fall short.

## 4.1 Concept Drift and Temporal Distribution Shift

The statistical properties of network traffic change over time. The FlowCLIP [4] study observed a performance drop from 81.38% to 69.55% within two weeks on the CESNET-

Table 1: Comparative Performance of Deep Learning Methods

| Method                   | Architecture               | Dataset                      | Performance               | Disadvantage                   |
|--------------------------|----------------------------|------------------------------|---------------------------|--------------------------------|
| ET-BERT [1]              | Transformer                | ISCX-Tor<br>CSTNET-TLS 1.3   | F1=99.2%<br>F1=97.4%      | Token-level limit              |
| MIETT [2]                | Transformer<br>(Two-Level) | CrossPlatform<br>CICIoT 2023 | Acc=93.0%<br>Acc=88.5%    | High latency                   |
| MalMoE [5]               | GNN + MoE                  | NF-CIC-IDS<br>(w/ drift)     | Acc=97.2%<br>F1=94.8%     | Graph construction<br>overhead |
| Transfer Learning<br>[3] | Universal<br>Embedding     | Multiple<br>(10 tasks)       | +6.4% avg<br>improvement  | Pre-training cost              |
| FlowCLIP [4]             | Contrastive<br>Learning    | CESNET-QUIC22<br>(Week 2)    | Acc=81.4%<br>(Week 2)     | Temporal<br>degradation        |
| HW-NAS-CNN [10]          | 1D-CNN<br>(Optimized)      | Multiple                     | Acc=96.6%<br>(88K params) | No temporal<br>modeling        |

QUIC22 dataset. All models tested (1-NN, XGBoost, FlowCLIP) exhibited similar declines, showing that concept drift is a model-agnostic problem. It arises from causes such as application updates, changes in user behavior, changing network conditions, and the evolution of TLS/QUIC implementations.

## 4.2 Zero-Day Traffic Detection

Detecting new applications not seen during training (zero-day) is a fundamental problem for supervised classification. Standard softmax-based classifiers are unable to distinguish classes they have not learned.

## 4.3 Rare Class Classification Problem

In network traffic, some applications are used very frequently while others are used very rarely. This leads to class imbalance in the training data.

The FlowCLIP [4] study clearly demonstrates this problem: while the model achieved an overall accuracy of 81.38%, its Macro F1 score was only 44.91%. The 36.47 percentage-point gap between these two metrics points to a critical issue.

Accuracy weights every sample equally, whereas Macro F1 weights every class equally. The larger the gap between the two, the more poorly the rare classes are being classified.

## 4.4 Real-Time Processing Overhead

Today’s high-speed networks process millions of network requests per second. However, the classification time of deep learning-based models introduces a latency of 20-200ms, which is an unacceptable delay for real-time applications.

## 4.5 Adversarial Vulnerability

Deep learning models are vulnerable to small, deliberate perturbations made to packets. This problem also exists in the field of encrypted traffic classification. An attacker can

Table 2: Model Inference Latency

| Model         | Inference Time |
|---------------|----------------|
| Random Forest | 0.1-1 ms       |
| 1D-CNN        | 1-5 ms         |
| LSTM          | 10-50 ms       |
| Transformer   | 50-200 ms      |
| GNN           | 100-500 ms     |

make imperceptible modifications to network traffic to cause a model to misclassify it. For example, malicious malware traffic can be classified as normal web traffic by adding minimal changes to packet sizes.

Such attack methods have been tested in the literature [6]. The **DeepFool** attack can be cited as the most effective example of this type of attack, capable of significantly reducing model accuracy.

An even more dangerous type of attack, UAP [7], involves a single perturbation vector that can be effective across many different inputs. In other words, instead of designing a custom attack for every sample, an attacker can mislead the entire model with a single generic perturbation. This substantially increases the scalability of such attacks.

## 5 Future Research Directions

The findings from the literature and the shortcomings of existing methods have been reviewed. The open directions on which future research in encrypted traffic analysis should focus are listed below.

### 5.1 Privacy-Preserving Training via Federated Learning

Because network traffic packets may contain users’ personal data, regulations such as Turkey’s Personal Data Protection Law (KVKK) and the GDPR prevent them from being collected in a central location. Federated Learning can be used to address this problem. In this approach, a central server initializes a global model and distributes it to each participating organization. Each organization trains this model on its own local data, computes model updates, and sends only these updates back to the central server. The central server aggregates the updates received from all organizations to produce a new global model, and this cycle continues iteratively. In this way, a shared model can be trained without the raw data ever leaving the organization, thereby preserving data privacy.

### 5.2 Few-Shot and Meta-Learning Approaches

Deep learning models require large amounts of labeled data for training. When they encounter data not represented in the training set, they may fail to classify it correctly. In such cases, network administrators must collect new data and retrain the model, a process that is time-consuming and costly. Few-shot learning approaches can be used to address this problem, as they allow a model to be trained from only a handful of examples.

## 6 Conclusion

With the widespread adoption of the TLS and QUIC protocols, encrypted traffic has increased substantially. Encrypting network traffic is of critical importance for both security and privacy. However, encrypted traffic analysis is still needed for traffic optimization, detection of malicious activity, and network management. For this reason, research in the field of encrypted network traffic analysis and classification is extremely valuable.

Even though network traffic is encrypted, it can still be analyzed using deep learning methods. In particular, Transformer-based methods such as ET-BERT and MIETT achieve F1 scores of 96-99%, delivering the best performance among the methods reviewed. Transfer learning and graph-based methods also stand out in the literature as highly promising approaches.

The inference latencies of Transformer architectures (50-200ms) and graph neural networks (100-500ms) create a bottleneck on terabit-speed network links. Faster and more efficient methods are therefore needed for real-time applications.

The ability of adversarial attacks such as DeepFool and UAP to mislead models with minimal perturbations highlights the risk of deploying deep learning architectures directly within security systems.

## References

- [1] Lin, X., Xiong, G., Gou, G., Li, Z., Shi, J., Yu, J. (2022). *ET-BERT: A Contextualized Datagram Representation with Pre-training Transformers for Encrypted Traffic Classification*. Proceedings of the ACM Web Conference 2022 (WWW '22). arXiv:2202.06335.
- [2] Zhao, J., et al. (2024). *MIETT: Multi-Instance Encrypted Traffic Transformer for Encrypted Traffic Classification*. AAAI Conference on Artificial Intelligence 2025. arXiv:2412.15306.
- [3] Chen, Y., et al. (2026). *Universal Embedding Function for Traffic Classification via QUIC Domain Recognition Pretraining: A Transfer Learning Success*. IEEE Transactions on Network and Service Management. arXiv:2502.12930.
- [4] Wang, H., et al. (2026). *FlowCLIP: Contrastive Learning for Encrypted Traffic Classification with Temporal Distribution Shift*. arXiv preprint arXiv:2606.17746.
- [5] Liu, S., et al. (2026). *MalMoE: Mixture-of-Experts Enhanced Encrypted Malicious Traffic Detection Under Graph Drift*. IEEE INFOCOM 2026 (Accepted). arXiv:2602.10157.
- [6] Maarouf, A., et al. (2021). *Evaluating Resilience of Encrypted Traffic Classification Against Adversarial Evasion Attacks*. IEEE Conference. arXiv:2105.14564.
- [7] Zhang, H., et al. (2020). *Universal Adversarial Perturbations for Deep Learning-based Network Traffic Classification*. IEEE Transactions on Network and Service Management. arXiv:2003.01261.
- [8] Yang, Z., et al. (2021). *Detection of Unknown Encrypted Malicious Traffic in Real Time via Flow Interaction Graph Analysis*. IEEE Transactions on Network and Service Management. arXiv:2104.03182.
- [9] Sharafaldin, I., Lashkari, A. H., Ghorbani, A. A. (2018). *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*. Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP).
- [10] Li, M., et al. (2025). *Hardware-Aware Neural Architecture Search for Encrypted Traffic Classification*. arXiv preprint arXiv:2506.11319.
- [11] Zhou, T., et al. (2025). *NetMamba: Efficient Network Traffic Classification with State Space Models*. arXiv preprint arXiv:2601.21792.
- [12] Kumar, R., et al. (2025). *NetSight: Self-Supervised Network Traffic Classification with Pseudo-Labeling*. arXiv preprint arXiv:2508.15100.
- [13] Google. (2024). *HTTPS Encryption on the Web - Transparency Report*. <https://transparencyreport.google.com/https/overview>